

Problema 01: Análise de Tensões em Trelças Planas

Diogo Estéfano Cambraia Freitas | Engenharia de Telecomunicações

LLM 1 | Chat GPT

```
# =====
# ANÁLISE DE TRELIÇAS PLANAS ISOSTÁTICAS
# Implementação completa em Python para Google Colab
#
# Requisitos atendidos:
# ✓ Vetores/listas de tamanho variável
# ✓ Método de Gauss-Jordan manual
# ✓ Cálculo de forças internas
# ✓ Classificação: tração/compressão/nula
# ✓ Estatísticas estruturais
# ✓ Tabelas de resultados
# ✓ Visualizações gráficas obrigatórias
#
# Autor: ChatGPT
# =====

import math
import matplotlib.pyplot as plt
import numpy as np

# =====
# FUNÇÃO: RESOLUÇÃO DO SISTEMA LINEAR (GAUSS-JORDAN MANUAL)
# =====

def gauss_jordan(A, b):

    n = len(A)

    # Monta matriz aumentada
    M = [A[i] + [b[i]] for i in range(n)]

    for i in range(n):

        # Pivotamento parcial
        max_row = i
        for k in range(i + 1, n):
            if abs(M[k][i]) > abs(M[max_row][i]):
```

```

        max_row = k

M[i], M[max_row] = M[max_row], M[i]

pivot = M[i][i]

if abs(pivot) < 1e-12:
    raise Exception("Sistema singular ou mal condicionado.")

# Normaliza linha do pivô
for j in range(i, n + 1):
    M[i][j] /= pivot

# Elimina demais linhas
for k in range(n):
    if k != i:
        fator = M[k][i]
        for j in range(i, n + 1):
            M[k][j] -= fator * M[i][j]

# Extrai solução
x = [M[i][n] for i in range(n)]

return x

```

```

# =====
# DEFINIÇÃO DA TRELIÇA
# (ALTERE OS DADOS PARA TESTAR DIFERENTES ESTRUTURAS)
# =====

# -----
# NÚMERO DE NÓS
# -----

N = 4

# -----
# COORDENADAS DOS NÓS
# -----

x = [0, 4, 8, 4]
y = [0, 0, 0, 3]

# -----
# BARRAS (conectividade)
# cada barra conecta [nó_i, nó_j]
# -----

```

```
barras = [  
    [0, 1],  
    [1, 2],  
    [0, 3],  
    [1, 3],  
    [2, 3]  
]
```

```
B = len(barras)
```

```
# -----  
# CARGAS EXTERNAS  
# -----
```

```
Fx = [0, 0, 0, 0]  
Fy = [0, 0, 0, -10000] # carga vertical no nó 3
```

```
# -----  
# APOIOS  
#  
# "livre"  
# "movel_y" -> restringe Y  
# "fixo"    -> restringe X e Y  
# -----
```

```
apoios = [  
    "fixo",  
    "livre",  
    "movel_y",  
    "livre"  
]
```

```
# =====  
# MONTAGEM DAS REAÇÕES DE APOIO  
# =====
```

```
reacoes = []
```

```
for i in range(N):
```

```
    if apoios[i] == "fixo":  
        reacoes.append((i, 'x'))  
        reacoes.append((i, 'y'))
```

```
    elif apoios[i] == "movel_y":  
        reacoes.append((i, 'y'))
```

```

R = len(reacoes)

# =====
# VERIFICAÇÃO ISOSTÁTICA
# =====

if B + R != 2 * N:
    print("ATENÇÃO:")
    print("A estrutura pode não ser isostática.")
    print(f"B + R = {B + R}")
    print(f"2N = {2*N}")

# =====
# MONTAGEM DA MATRIZ DE EQUILÍBRIO
# =====

num_incognitas = B + R
num_equacoes = 2 * N

A = [[0.0 for _ in range(num_incognitas)] for _ in range(num_equacoes)]
b = [0.0 for _ in range(num_equacoes)]

# -----
# CONTRIBUIÇÃO DAS BARRAS
# -----

for idx_barra, (i, j) in enumerate(barras):

    dx = x[j] - x[i]
    dy = y[j] - y[i]

    L = math.sqrt(dx**2 + dy**2)

    cx = dx / L
    cy = dy / L

    # Equações do nó i
    A[2*i][idx_barra] += cx
    A[2*i + 1][idx_barra] += cy

    # Equações do nó j
    A[2*j][idx_barra] -= cx
    A[2*j + 1][idx_barra] -= cy

# -----
# CONTRIBUIÇÃO DAS REAÇÕES
# -----

```

```
for idx_reac, (no, direcao) in enumerate(reacoes):
```

```
    col = B + idx_reac
```

```
    if direcao == 'x':
```

```
        A[2*no][col] = 1.0
```

```
    elif direcao == 'y':
```

```
        A[2*no + 1][col] = 1.0
```

```
# -----
```

```
# VETOR DE CARGAS
```

```
# -----
```

```
for i in range(N):
```

```
    b[2*i] = -Fx[i]
```

```
    b[2*i + 1] = -Fy[i]
```

```
# =====
```

```
# RESOLUÇÃO DO SISTEMA
```

```
# =====
```

```
solucao = gauss_jordan(A, b)
```

```
# =====
```

```
# EXTRAÇÃO DAS FORÇAS NAS BARRAS
```

```
# =====
```

```
forcas_barras = solucao[:B]
```

```
forcas_reacoes = solucao[B:]
```

```
# =====
```

```
# CLASSIFICAÇÃO DAS BARRAS
```

```
# =====
```

```
classificacao = []
```

```
for F in forcas_barras:
```

```
    if F > 1e-6:
```

```
        classificacao.append("TRAÇÃO")
```

```
    elif F < -1e-6:
```

```
        classificacao.append("COMPRESSÃO")
```

```
    else:
```

```
        classificacao.append("NULA")
```

```

# =====
# ESTATÍSTICAS
# =====

magnitudes = [abs(F) for F in forcas_barras]

barra_mais_solicitada = magnitudes.index(max(magnitudes))

forca_media = sum(magnitudes) / B

soma_reacoes = sum(abs(r) for r in forcas_reacoes)

# =====
# TABELA DE RESULTADOS
# =====

print("\n" + "="*70)
print("RESULTADOS DAS BARRAS")
print("="*70)

print(f'{"Barra":<10}{ "Força (N)":<20}{ "Classificação"}')

for i in range(B):
    print(f'{i:<10}{forcas_barras[i]:<20.2f}{classificacao[i]}')

print("\n" + "="*70)
print("ESTATÍSTICAS")
print("="*70)

print(f"Barra mais solicitada : {barra_mais_solicitada}")
print(f"Força máxima          : {max(magnitudes):.2f} N")
print(f"Força média            : {forca_media:.2f} N")
print(f"Soma das reações       : {soma_reacoes:.2f} N")

# =====
# FUNÇÃO AUXILIAR PARA DESENHO DOS APOIOS
# =====

def desenhar_apoio(ax, x0, y0, tipo):

    if tipo == "fixo":
        ax.plot(x0, y0, marker='s', markersize=14, color='black')

    elif tipo == "movel_y":
        ax.plot(x0, y0, marker='^', markersize=14, color='green')

# =====

```

```

# GRÁFICO 1
# TRELIÇA ORIGINAL
# =====

plt.figure(figsize=(10, 6))

for idx, (i, j) in enumerate(barras):

    plt.plot(
        [x[i], x[j]],
        [y[i], y[j]],
        'k-',
        linewidth=2
    )

# nós
for i in range(N):

    plt.plot(x[i], y[i], 'bo')

    plt.text(
        x[i] + 0.1,
        y[i] + 0.1,
        f'{i}',
        fontsize=12,
        color='blue'
    )

    desenhar_apoio(plt.gca(), x[i], y[i], apoios[i])

plt.title("Trelença Original")
plt.xlabel("X")
plt.ylabel("Y")
plt.axis('equal')
plt.grid(True)

plt.show()

# =====
# GRÁFICO 2
# MAPA DE CALOR DAS FORÇAS
# =====

plt.figure(figsize=(10, 6))

Fmax = max(abs(f) for f in forcas_barras)

for idx, (i, j) in enumerate(barras):

```

```

F = forcas_barras[idx]

intensidade = abs(F) / Fmax

# Azul = tração
# Vermelho = compressão
if F >= 0:
    cor = (0, 0, intensidade)

else:
    cor = (intensidade, 0, 0)

plt.plot(
    [x[i], x[j]],
    [y[i], y[j]],
    color=cor,
    linewidth=5
)

# nós
for i in range(N):

    plt.plot(x[i], y[i], 'ko')

    plt.text(
        x[i] + 0.1,
        y[i] + 0.1,
        f'{i}',
        fontsize=11
    )

plt.title("Mapa de Calor das Forças Internas")
plt.xlabel("X")
plt.ylabel("Y")
plt.axis('equal')
plt.grid(True)

plt.show()

# =====
# GRÁFICO 3
# BARRAS DAS FORÇAS
# =====

plt.figure(figsize=(10, 5))

cores = []

```



```

for F in forcas_barras:

    if F > 0:
        cores.append('blue')

    elif F < 0:
        cores.append('red')

    else:
        cores.append('gray')

plt.bar(
    range(B),
    forcas_barras,
    color=cores
)

plt.axhline(0, color='black')

plt.title("Magnitude das Forças por Barra")
plt.xlabel("Barra")
plt.ylabel("Força (N)")
plt.grid(True)

plt.show()

# =====
# GRÁFICO 4
# HISTOGRAMA DAS FORÇAS
# =====

plt.figure(figsize=(10, 5))

plt.hist(
    forcas_barras,
    bins=8,
    color='orange',
    edgecolor='black'
)

plt.title("Histograma das Forças Internas")
plt.xlabel("Força (N)")
plt.ylabel("Frequência")
plt.grid(True)

plt.show()

```

```
# =====  
# FIM DO PROGRAMA  
# =====
```

Correção/ código final Chat GPT

```
# =====  
# ANÁLISE DE TRELIÇAS PLANAS — GOOGLE COLAB  
# INTERFACE WEB + TODOS OS GRÁFICOS OBRIGATÓRIOS  
# =====
```

```
import math  
import ast  
import matplotlib.pyplot as plt  
import ipywidgets as widgets
```

```
from IPython.display import display, clear_output
```

```
# =====  
# GAUSS-JORDAN MANUAL  
# =====
```

```
def gauss_jordan(A, b):
```

```
    n = len(A)
```

```
    M = [A[i] + [b[i]] for i in range(n)]
```

```
    for i in range(n):
```

```
        max_row = i
```

```
        for k in range(i + 1, n):
```

```
            if abs(M[k][i]) > abs(M[max_row][i]):  
                max_row = k
```

```
    M[i], M[max_row] = M[max_row], M[i]
```

```
    pivot = M[i][i]
```

```
    if abs(pivot) < 1e-12:  
        raise Exception("Sistema singular.")
```

```
    for j in range(i, n + 1):  
        M[i][j] /= pivot
```

```
    for k in range(n):
```

```

        if k != i:

            fator = M[k][i]

            for j in range(i, n + 1):
                M[k][j] -= fator * M[i][j]

    return [M[i][n] for i in range(n)]

# =====
# DESENHO APOIOS
# =====

def desenhar_apoio(ax, x, y, tipo):

    if tipo == "fixo":

        ax.plot(
            x,
            y,
            marker='s',
            markersize=12,
            color='black'
        )

    elif tipo == "movel_y":

        ax.plot(
            x,
            y,
            marker='^',
            markersize=12,
            color='green'
        )

# =====
# FUNÇÃO PRINCIPAL
# =====

def analisar_trelica(
    titulo,
    N,
    x,
    y,
    barras,
    Fx,
    Fy,

```

```

apoios
):

B = len(barras)

# =====
# REAÇÕES
# =====

reacoes = []

for i in range(N):

    if apoios[i] == "fixo":

        reacoes.append((i, 'x'))
        reacoes.append((i, 'y'))

    elif apoios[i] == "movel_y":

        reacoes.append((i, 'y'))

# =====
# MATRIZ
# =====

incognitas = B + len(reacoes)

A = [[0.0 for _ in range(incognitas)] for _ in range(2*N)]

b = [0.0 for _ in range(2*N)]

# =====
# BARRAS
# =====

for idx, (i, j) in enumerate(barras):

    dx = x[j] - x[i]
    dy = y[j] - y[i]

    L = math.sqrt(dx**2 + dy**2)

    cx = dx / L
    cy = dy / L

    A[2*i][idx] += cx
    A[2*i+1][idx] += cy

```

```

    A[2*j][idx] -= cx
    A[2*j+1][idx] -= cy

# =====
# REAÇÕES
# =====

for idx, (no, direcao) in enumerate(reacoes):

    col = B + idx

    if direcao == 'x':
        A[2*no][col] = 1

    else:
        A[2*no+1][col] = 1

# =====
# CARGAS
# =====

for i in range(N):

    b[2*i] = -Fx[i]
    b[2*i+1] = -Fy[i]

# =====
# SOLUÇÃO
# =====

solucao = gauss_jordan(A, b)

forcas = solucao[:B]

# =====
# CLASSIFICAÇÃO
# =====

classes = []

for F in forcas:

    if F > 1e-6:
        classes.append("TRAÇÃO")

    elif F < -1e-6:
        classes.append("COMPRESSÃO")

```

```

else:
    classes.append("NULA")

# =====
# RESULTADOS
# =====

print("\n")
print("="*70)
print(titulo)
print("="*70)

print(f"{'Barra':<10}{'Força(N)':<20}{'Classificação'}")

for i in range(B):

    print(f"{'i':<10}{'forças[i]':<20.2f}{'classes[i]'}")

# =====
# ESTATÍSTICAS
# =====

magnitudes = [abs(f) for f in forcas]

print("\nESTATÍSTICAS")
print("-"*70)

print("Barra mais solicitada:", magnitudes.index(max(magnitudes)))
print("Força máxima:", max(magnitudes))
print("Força média:", sum(magnitudes)/B)

# =====
# 1) TRELIÇA ORIGINAL
# =====

plt.figure(figsize=(10,6))

for (i, j) in barras:

    plt.plot(
        [x[i], x[j]],
        [y[i], y[j]],
        'k-',
        linewidth=2
    )

for i in range(N):

```

```

plt.plot(x[i], y[i], 'bo')

plt.text(
    x[i] + 0.1,
    y[i] + 0.1,
    str(i),
    fontsize=11
)

desenhar_apoio(
    plt.gca(),
    x[i],
    y[i],
    apoios[i]
)

plt.title(f'{titulo} — Trelença Original')

plt.grid(True)

plt.axis('equal')

plt.show()

# =====
# 2) MAPA DE CALOR
# =====

plt.figure(figsize=(10,6))

Fmax = max(abs(f) for f in forcas)

for idx, (i, j) in enumerate(barras):

    F = forcas[idx]

    intensidade = abs(F) / Fmax

    # Azul = Tração
    # Vermelho = Compressão

    if F >= 0:

        cor = (0, 0, intensidade)

    else:

```

```

        cor = (intensidade, 0, 0)

    plt.plot(
        [x[i], x[j]],
        [y[i], y[j]],
        color=cor,
        linewidth=6
    )

for i in range(N):

    plt.plot(x[i], y[i], 'ko')

plt.title(f"{titulo} — Mapa de Calor")

plt.grid(True)

plt.axis('equal')

plt.show()

# =====
# 3) GRÁFICO DE BARRAS
# =====

cores = []

for F in forcas:

    if F > 0:
        cores.append("blue")

    elif F < 0:
        cores.append("red")

    else:
        cores.append("gray")

plt.figure(figsize=(10,5))

plt.bar(
    range(B),
    forcas,
    color=cores
)

plt.axhline(0, color='black')

```



```

plt.title(f"{titulo} — Forças nas Barras")

plt.xlabel("Barra")

plt.ylabel("Força (N)")

plt.grid(True)

plt.show()

# =====
# 4) HISTOGRAMA
# =====

plt.figure(figsize=(10,5))

plt.hist(
    forcas,
    bins=8,
    color='orange',
    edgecolor='black'
)

plt.title(f"{titulo} — Histograma das Forças")

plt.xlabel("Força")

plt.ylabel("Frequência")

plt.grid(True)

plt.show()

# =====
# INTERFACE WEB
# =====

entrada = widgets.Textarea(
    value="",
    placeholder='Cole os dados do gerador aqui...',
    layout=widgets.Layout(width='100%', height='300px')
)

botao = widgets.Button(
    description='ANALISAR TRELIÇA',
    button_style='success'
)

```

```

saida = widgets.Output()

# =====
# EVENTO
# =====

def executar(b):

    with saida:

        clear_output()

        try:

            texto = entrada.value

            linhas = texto.split("\n")

            dados = {}

            for linha in linhas:

                if "=" in linha:

                    chave, valor = linha.split("=", 1)

                    chave = chave.strip()
                    valor = valor.strip()

                    dados[chave] = ast.literal_eval(valor)

            # =====
            # TRELIÇA USUÁRIO
            # =====

            analisar_trelica(
                "TRELIÇA DO USUÁRIO",
                dados["N"],
                dados["x"],
                dados["y"],
                dados["barras"],
                dados["Fx"],
                dados["Fy"],
                dados["apoios"]
            )

            # =====
            # TRELIÇA WARREN

```

```
# =====
```

```
Nw = 6
```

```
xw = [0,2,4,6,8,10]
```

```
yw = [0,2,0,2,0,2]
```

```
barras_w = [
```

```
    [0,1],
```

```
    [1,2],
```

```
    [2,3],
```

```
    [3,4],
```

```
    [4,5],
```

```
    [0,2],
```

```
    [2,4],
```

```
    [1,3],
```

```
    [3,5]
```

```
]
```

```
Fxw = [0]*Nw
```

```
Fyw = [0,0,-10000,0,-10000,0]
```

```
apoios_w = [
```

```
    "fixo",
```

```
    "livre",
```

```
    "livre",
```

```
    "livre",
```

```
    "livre",
```

```
    "movel_y"
```

```
]
```

```
analisar_trelica(
```

```
    "TRELIÇA WARREN",
```

```
    Nw,
```

```
    xw,
```

```
    yw,
```

```
    barras_w,
```

```
    Fxw,
```

```
    Fyw,
```

```
    apoios_w
```

```
)
```

```
except Exception as e:
```

```
    print("ERRO:")
```

```
    print(e)
```

```
# =====
# BOTÃO
# =====
```

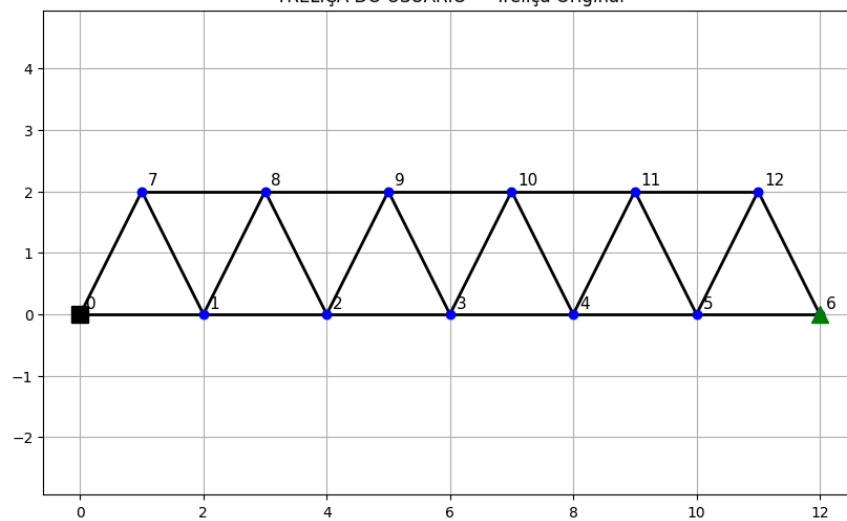
```
botao.on_click(executar)
```

```
# =====
# EXIBIÇÃO
# =====
```

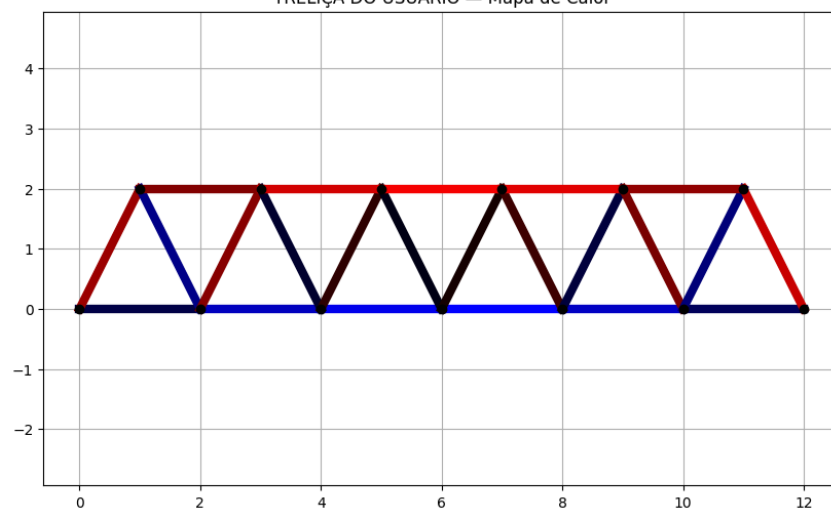
```
display(entrada)
display(botao)
display(saida)
```

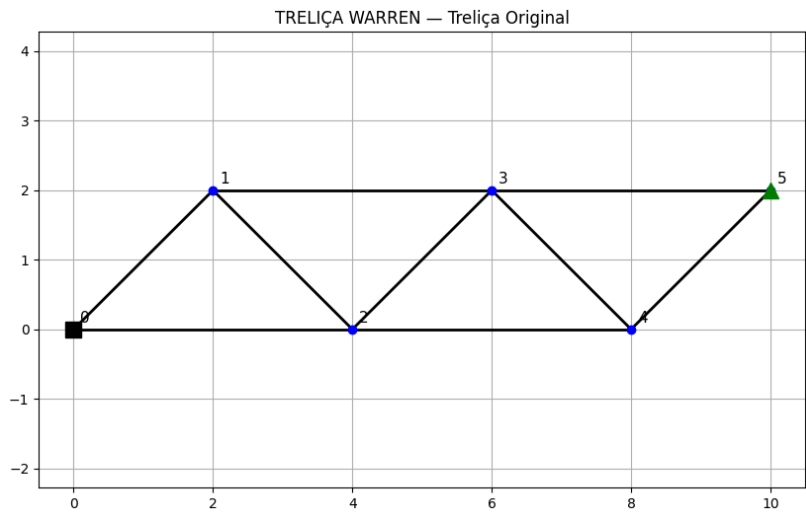
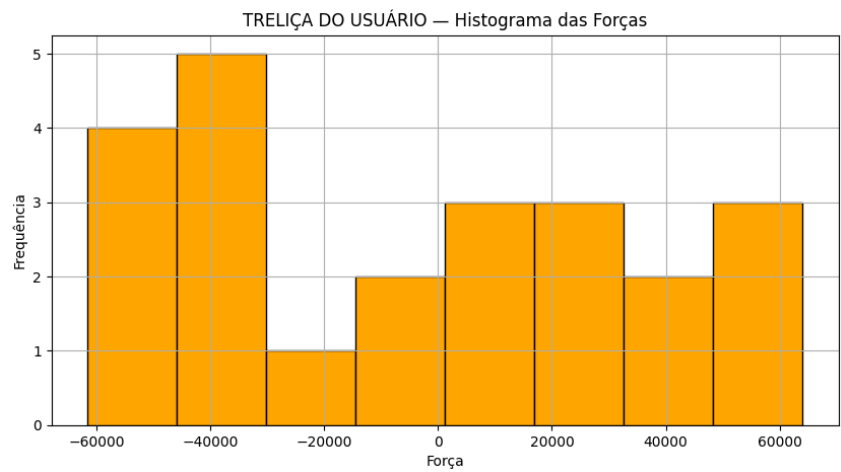
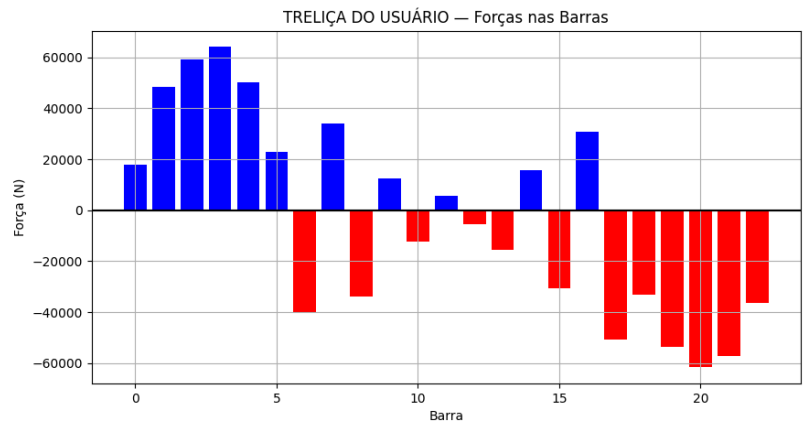
Anexos (finais) Chat GPT

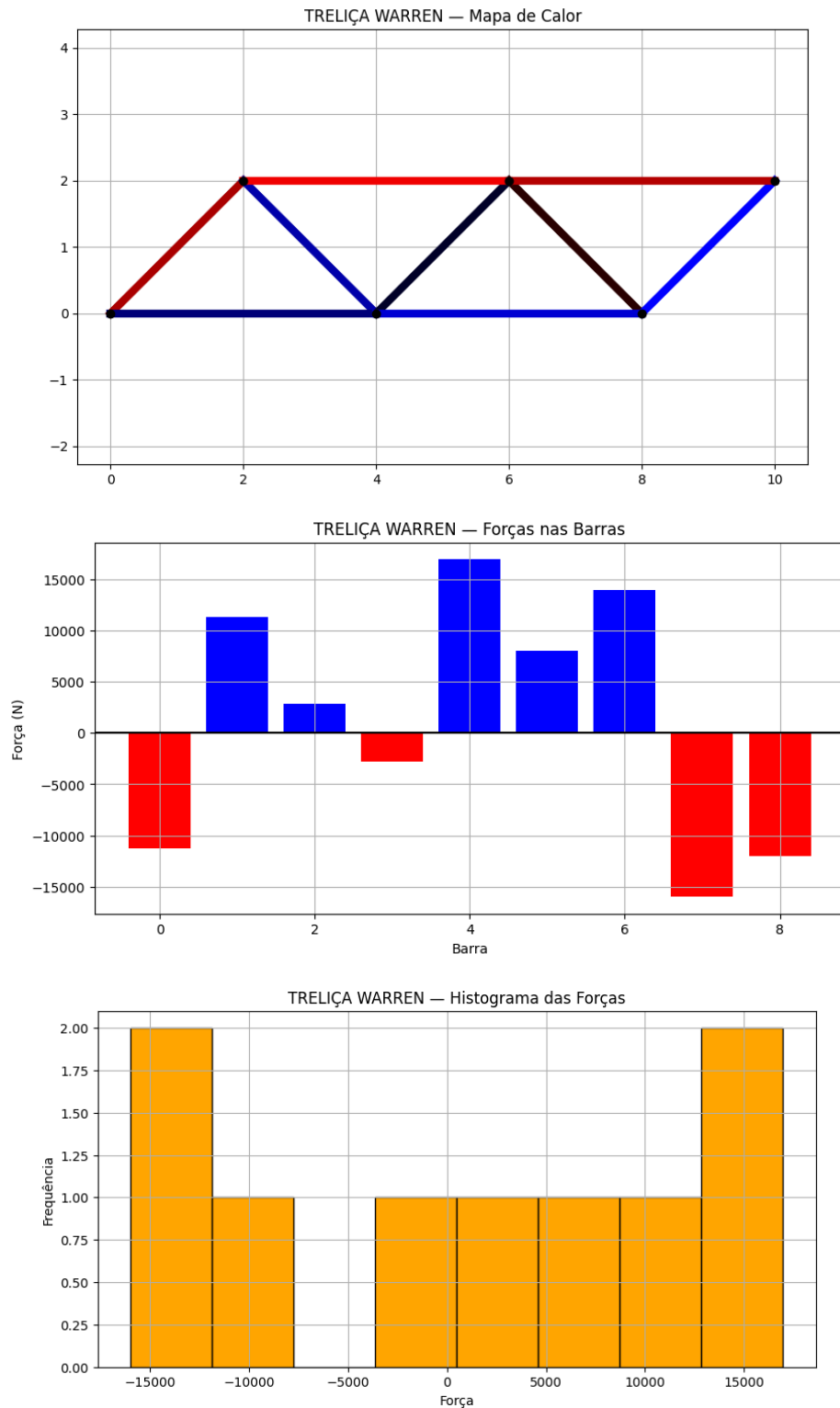
TRELIÇA DO USUÁRIO — Treliza Original



TRELIÇA DO USUÁRIO — Mapa de Calor







LLM 2 | Copilot

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import ipywidgets as widgets
from IPython.display import display
```

```

# -----
# Método de Gauss-Jordan
# -----
def gauss_jordan(A, b):
    n = len(b)
    M = np.hstack([A.astype(float), b.reshape(-1,1)])
    for i in range(n):
        if abs(M[i,i]) < 1e-12:
            for j in range(i+1,n):
                if abs(M[j,i]) > 1e-12:
                    M[[i,j]] = M[[j,i]]
                    break
            if abs(M[i,i]) < 1e-12:
                continue
            M[i] = M[i] / M[i,i]
            for j in range(n):
                if j != i:
                    M[j] = M[j] - M[j,i]*M[i]
    return M[:, -1]

```

```

# -----
# Função principal
# -----
def analisar_trelica(N, x, y, barras, Fx, Fy, apoios):
    B = len(barras)
    dof = 2*N
    A = np.zeros((dof, B + dof))
    b = np.zeros(dof)

    for i in range(N):
        b[2*i] = Fx[i]
        b[2*i+1] = Fy[i]

    for k, (i,j) in enumerate(barras):
        dx = x[j]-x[i]
        dy = y[j]-y[i]
        L = np.sqrt(dx**2+dy**2)
        if L == 0: continue
        cx, cy = dx/L, dy/L
        A[2*i, k] = cx
        A[2*i+1, k] = cy
        A[2*j, k] = -cx
        A[2*j+1, k] = -cy

    reacoes = []
    for i, tipo in enumerate(apoios):
        if tipo == 'fixo':

```

```

        A[2*i, B+len(reacoes)] = 1; reacoes.append(f"x{i}")
        A[2*i+1, B+len(reacoes)] = 1; reacoes.append(f"y{i}")
    elif tipo == 'movel_x':
        A[2*i, B+len(reacoes)] = 1; reacoes.append(f"x{i}")
    elif tipo == 'movel_y':
        A[2*i+1, B+len(reacoes)] = 1; reacoes.append(f"y{i}")

try:
    f = gauss_jordan(A, b)
except Exception as e:
    print("Erro ao resolver sistema:", e)
    return None, None

F_barras = f[:B]

classificacao = []
for val in F_barras:
    if val > 1e-6: classificacao.append("Tração")
    elif val < -1e-6: classificacao.append("Compressão")
    else: classificacao.append("Nula")

barra_max = np.argmax(np.abs(F_barras)) if len(F_barras)>0 else None
stats = {
    "Barra mais solicitada": barra_max,
    "Força máxima": F_barras[barra_max] if barra_max is not None else None,
    "Força média": np.mean(F_barras) if len(F_barras)>0 else None,
    "Somatório reações": np.sum(f[B:]) if len(f)>B else None
}

df = pd.DataFrame({
    "Barra": range(B),
    "Nós": barras,
    "Força": F_barras,
    "Classificação": classificacao
})

print(df)
print("\nEstatísticas:", stats)

plt.figure(figsize=(6,6))
for (i,j) in barras:
    plt.plot([x[i],x[j]], [y[i],y[j]], 'k-')
plt.scatter(x,y,c='black')
for i,(xi,yi) in enumerate(zip(x,y)):
    plt.text(xi,yi,f"{i}",color='red')
plt.title("Treliza Original")
plt.show()

```



```

plt.figure(figsize=(6,6))
for k,(i,j) in enumerate(barras):
    cor = 'blue' if F_barras[k]>0 else 'red'
    plt.plot([x[i],x[j]], [y[i],y[j]],color=cor,
             linewidth=2+abs(F_barras[k])/max(abs(F_barras)+[1])*3)
plt.scatter(x,y,c='black')
plt.title("Treliza Colorida por Força")
plt.show()

```

```

plt.figure()
plt.bar(range(B),F_barras)
plt.title("Forças em cada barra")
plt.xlabel("Barra")
plt.ylabel("Força")
plt.show()

```

```

plt.figure()
plt.hist(F_barras[~np.isnan(F_barras)],bins=10)
plt.title("Distribuição das forças internas")
plt.show()

```

```

return df, stats

```

```

# -----
# Área de input interativo
# -----
entrada_widget = widgets.Textarea(
    value="",
    placeholder='Cole aqui o bloco de dados (N, x, y, barras, Fx, Fy, apoios)...',
    description='Input:',
    layout=widgets.Layout(width='100%', height='200px')
)
display(entrada_widget)

```

```

def executar_treliza(_):
    entrada = entrada_widget.value
    exec(entrada)
    df_user, stats_user = analisar_treliza(N, x, y, barras, Fx, Fy, apoios)

```

```

# Caso fixo: Treliza Warren
print("\n--- Treliza Warren ---")
Nw = 4
xw = [0, 2, 4, 6]
yw = [0, 0, 0, 0]
barras_w = [[0,1],[1,2],[2,3],[0,2],[2,1]] # 5 barras
Fwx = [0,0,0,0]
Fyw = [0,0,-1000,0] # carga no nó central inferior
apoios_w = ['fixo','livre','livre','movel_y']

```

```

df_warren, stats_warren = analisar_trelica(Nw, xw, yw, barras_w, Fxw, Fyw, apoios_w)

botao = widgets.Button(description="Executar análise")
botao.on_click(executar_trelica)
display(botao)

```

Correção/ código final Copilot

```

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import ipywidgets as widgets
from IPython.display import display

# -----
# Método de Gauss-Jordan
# -----
def gauss_jordan(A, b):
    n = len(b)
    M = np.hstack([A.astype(float), b.reshape(-1,1)])
    for i in range(n):
        if abs(M[i,i]) < 1e-12:
            for j in range(i+1,n):
                if abs(M[j,i]) > 1e-12:
                    M[[i,j]] = M[[j,i]]
                    break
            if abs(M[i,i]) < 1e-12:
                continue
            M[i] = M[i] / M[i,i]
            for j in range(n):
                if j != i:
                    M[j] = M[j] - M[j,i]*M[i]
    return M[:, -1]

# -----
# Função principal
# -----
def analisar_trelica(N, x, y, barras, Fx, Fy, apoios):
    B = len(barras)
    dof = 2*N
    A = np.zeros((dof, B + dof))
    b = np.zeros(dof)

    for i in range(N):
        b[2*i] = Fx[i]

```

```
b[2*i+1] = Fy[i]
```

```
for k, (i,j) in enumerate(barras):
```

```
    dx = x[j]-x[i]
```

```
    dy = y[j]-y[i]
```

```
    L = np.sqrt(dx**2+dy**2)
```

```
    if L == 0: continue
```

```
    cx, cy = dx/L, dy/L
```

```
    A[2*i, k] = cx
```

```
    A[2*i+1, k] = cy
```

```
    A[2*j, k] = -cx
```

```
    A[2*j+1, k] = -cy
```

```
reacoes = []
```

```
for i, tipo in enumerate(apoios):
```

```
    if tipo == 'fixo':
```

```
        A[2*i, B+len(reacoes)] = 1; reacoes.append(f"x{i}")
```

```
        A[2*i+1, B+len(reacoes)] = 1; reacoes.append(f"y{i}")
```

```
    elif tipo == 'movel_x':
```

```
        A[2*i, B+len(reacoes)] = 1; reacoes.append(f"x{i}")
```

```
    elif tipo == 'movel_y':
```

```
        A[2*i+1, B+len(reacoes)] = 1; reacoes.append(f"y{i}")
```

```
try:
```

```
    f = gauss_jordan(A, b)
```

```
except Exception as e:
```

```
    print("Erro ao resolver sistema:", e)
```

```
    return None, None
```

```
F_barras = f[:B]
```

```
classificacao = []
```

```
for val in F_barras:
```

```
    if val > 1e-6: classificacao.append("Tração")
```

```
    elif val < -1e-6: classificacao.append("Compressão")
```

```
    else: classificacao.append("Nula")
```

```
barra_max = np.argmax(np.abs(F_barras)) if len(F_barras)>0 else None
```

```
stats = {
```

```
    "Barra mais solicitada": barra_max,
```

```
    "Força máxima": F_barras[barra_max] if barra_max is not None else None,
```

```
    "Força média": np.mean(F_barras) if len(F_barras)>0 else None,
```

```
    "Somatório reações": np.sum(f[B:]) if len(f)>B else None
```

```
}
```

```
df = pd.DataFrame({
```

```
    "Barra": range(B),
```

```
    "Nós": barras,
```

```

        "Força": F_barras,
        "Classificação": classificacao
    })

print(df)
print("\nEstatísticas:", stats)

plt.figure(figsize=(6,6))
for (i,j) in barras:
    plt.plot([x[i],x[j]], [y[i],y[j]], 'k-')
plt.scatter(x,y,c='black')
for i,(xi,yi) in enumerate(zip(x,y)):
    plt.text(xi,yi,f"{i}",color='red')
plt.title("Trelença Original")
plt.show()

plt.figure(figsize=(6,6))
for k,(i,j) in enumerate(barras):
    cor = 'blue' if F_barras[k]>0 else 'red'
    plt.plot([x[i],x[j]], [y[i],y[j]], color=cor,
             linewidth=2+abs(F_barras[k])/max(abs(F_barras)+[1])*3)
plt.scatter(x,y,c='black')
plt.title("Trelença Colorida por Força")
plt.show()

plt.figure()
plt.bar(range(B),F_barras)
plt.title("Forças em cada barra")
plt.xlabel("Barra")
plt.ylabel("Força")
plt.show()

plt.figure()
plt.hist(F_barras[~np.isnan(F_barras)],bins=10)
plt.title("Distribuição das forças internas")
plt.show()

return df, stats

# -----
# Área de input interativo
# -----
entrada_widget = widgets.Textarea(
    value="",
    placeholder='Cole aqui o bloco de dados (N, x, y, barras, Fx, Fy, apoios)...',
    description='Input:',
    layout=widgets.Layout(width='100%', height='200px')
)

```

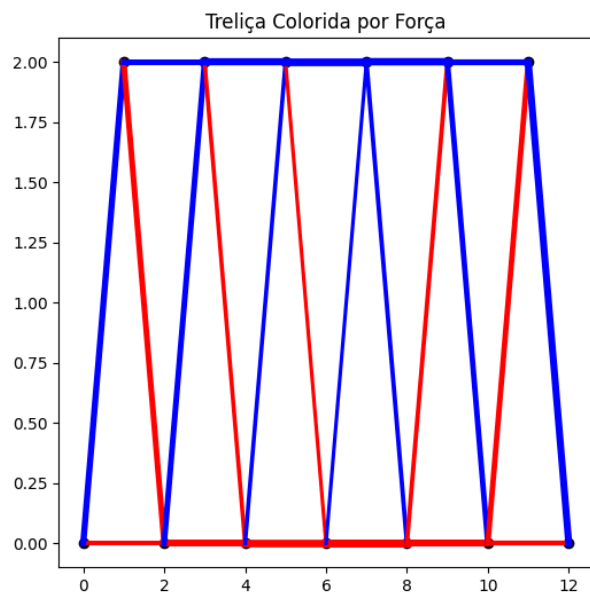
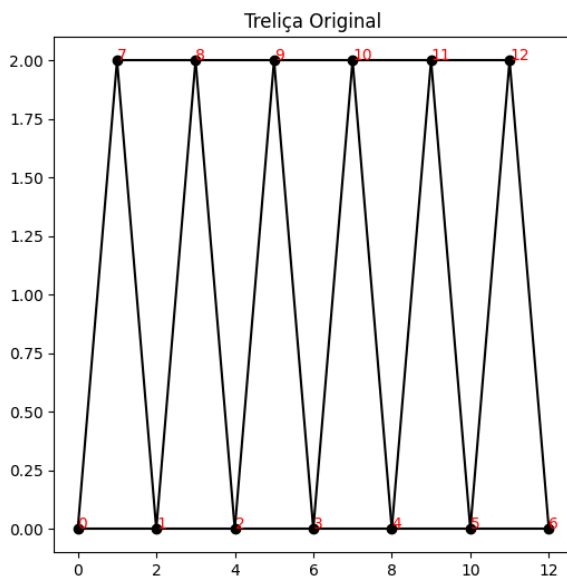
```
display(entrada_widget)
```

```
def executar_trelica(_):
    entrada = entrada_widget.value
    exec(entrada)
    df_user, stats_user = analisar_trelica(N, x, y, barras, Fx, Fy, apoios)

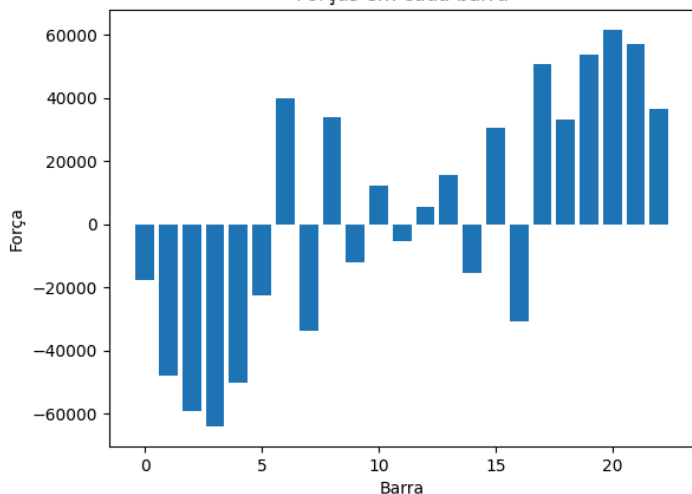
    # Caso fixo: Treliça Warren correta
    print("\n--- Treliça Warren ---")
    Nw = 4
    xw = [0, 2, 4, 6]
    yw = [0, 2, 0, 0]
    barras_w = [[0,1],[1,2],[2,3],[0,2],[1,3]]
    Fxw = [0,0,0,0]
    Fyw = [0,0,-1000,0] # carga vertical no nó central inferior
    apoios_w = ['fixo','livre','livre','movel_y']
    df_warren, stats_warren = analisar_trelica(Nw, xw, yw, barras_w, Fxw, Fyw, apoios_w)

botao = widgets.Button(description="Executar análise")
botao.on_click(executar_trelica)
display(botao)
```

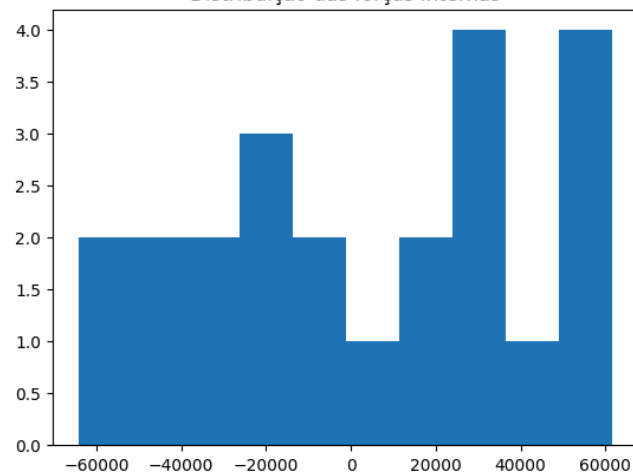
Anexos Copilot



Forças em cada barra

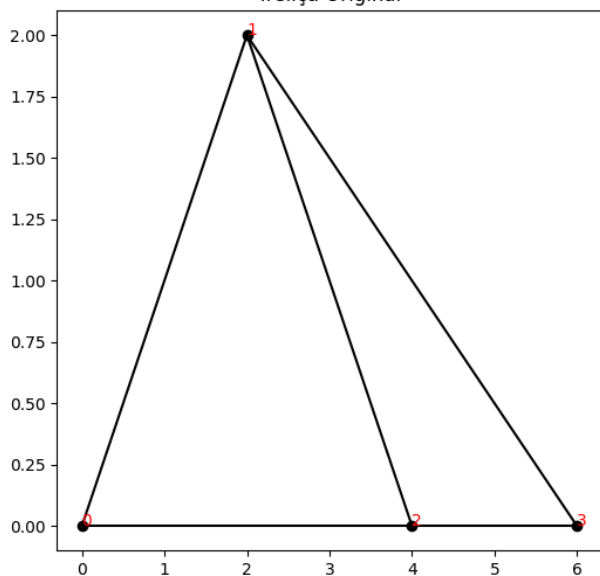


Distribuição das forças internas

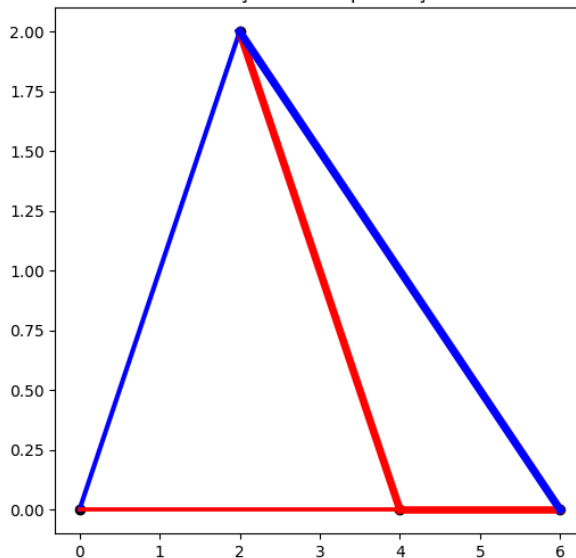


Treliça warren por copilot:

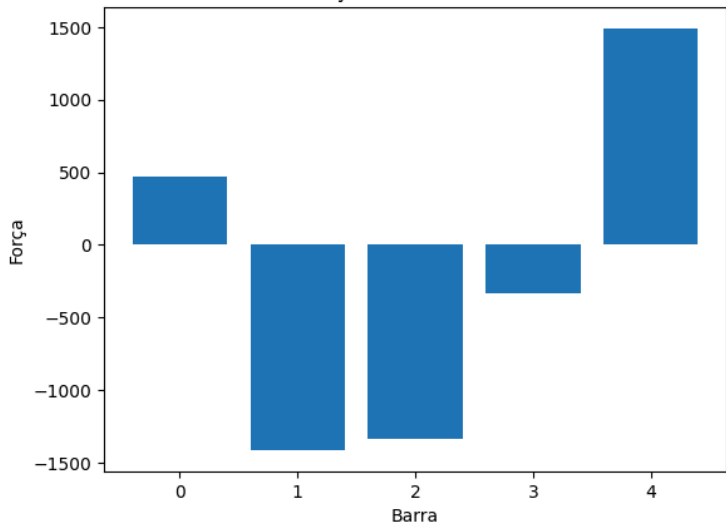
Treliça Original



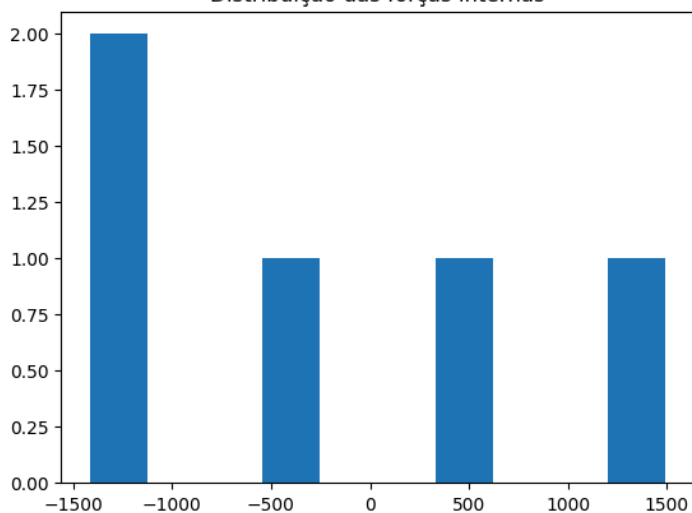
Treliça Colorida por Força



Forças em cada barra



Distribuição das forças internas



LLM 3 | Gemini

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import math
from ipywidgets import widgets
from IPython.display import display, clear_output

# --- FUNÇÕES DE CÁLCULO ---
def resolver_sistema_gauss(A, B):
    n = len(B)
    M = [A[i] + [B[i]] for i in range(n)]
    for i in range(n):
        max_el = abs(M[i][i])
        max_row = i
        for k in range(i + 1, n):
            if abs(M[k][i]) > max_el:
                max_el = abs(M[k][i]); max_row = k
        M[i], M[max_row] = M[max_row], M[i]
        for k in range(i + 1, n):
            if abs(M[i][i]) < 1e-12: continue
            c = -M[k][i] / M[i][i]
            for j in range(i, n + 1):
                M[k][j] = 0 if i == j else M[k][j] + c * M[i][j]
    x_sol = [0 for i in range(n)]
    for i in range(n - 1, -1, -1):
        if abs(M[i][i]) > 1e-12:
            x_sol[i] = M[i][n] / M[i][i]
            for k in range(i - 1, -1, -1):
                M[k][n] -= M[k][i] * x_sol[i]
    return x_sol

def analisar_trelica(N, x, y, barras, Fx, Fy, apoios):
    num_barras = len(barras)
    reacoes = []
    for i, tipo in enumerate(apoios):
        if tipo == 'fixo': reacoes.extend([(i, 'x'), (i, 'y')])
        elif tipo == 'movel_x': reacoes.append((i, 'x'))
        elif tipo == 'movel_y': reacoes.append((i, 'y'))
```

```

total_incognitas = num_barras + len(reacoes)
A = [[0.0 for _ in range(total_incognitas)] for _ in range(2 * N)]
B_vec = [0.0 for _ in range(2 * N)]

for i, (n1, n2) in enumerate(barras):
    dx, dy = x[n2] - x[n1], y[n2] - y[n1]
    L = math.sqrt(dx**2 + dy**2)
    cos, sen = dx / L, dy / L
    A[2*n1][i] += cos; A[2*n1+1][i] += sen
    A[2*n2][i] -= cos; A[2*n2+1][i] -= sen

for j, (no, eixo) in enumerate(reacoes):
    col = num_barras + j
    A[2*no + (0 if eixo == 'x' else 1)][col] = 1.0

for i in range(N):
    B_vec[2*i], B_vec[2*i+1] = -Fx[i], -Fy[i]

sol = resolver_sistema_gauss(A, B_vec)
return sol[:num_barras], sol[num_barras:], reacoes

def plotar_tudo(N, x, y, barras, forcas, titulo):
    fig, axs = plt.subplots(2, 2, figsize=(15, 10))

    # 1. Treliza Colorida
    max_f = max([abs(f) for f in forcas]) if forcas else 1
    for i, (n1, n2) in enumerate(barras):
        f = forcas[i]
        cor = 'blue' if f > 0.1 else 'red' if f < -0.1 else 'gray'
        lw = 1 + 4 * (abs(f)/max_f)
        axs[0,0].plot([x[n1], x[n2]], [y[n1], y[n2]], color=cor, linewidth=lw)
        axs[0,0].text((x[n1]+x[n2])/2, (y[n1]+y[n2])/2, f'{f:.0f}', fontsize=7)
    axs[0,0].scatter(x, y, color='black', s=30)
    axs[0,0].set_title(f'Estrutura: {titulo}\n(Azul: Tração | Vermelho: Compr.)')

    # 2. Magnitudes
    axs[0,1].bar(range(len(forcas)), [abs(f) for f in forcas], color='skyblue')
    axs[0,1].set_title("Magnitude das Forças por Barra")

    # 3. Histograma
    axs[1,0].hist(forcas, bins=15, color='purple', alpha=0.7)
    axs[1,0].set_title("Distribuição de Tensões")

    # 4. Tabela (Gráfica)
    axs[1,1].axis('off')
    dados = [[f'{b[0]}-{b[1]}', f'{f:.2f}', "T" if f>0.1 else "C" if f<-0.1 else "N" for b, f in
zip(barras, forcas)]

```



```

        tabela = axs[1,1].table(cellText=dados[:20], colLabels=['Barra', 'Força (N)', 'Tipo'],
loc='center') # mostra só as 20 primeiras
        axs[1,1].set_title("Tabela de Resultados (Top 20)")

plt.tight_layout()
plt.show()

# --- INTERFACE NO COLAB ---
txt_input = widgets.Textarea(
    placeholder='Cole seus dados aqui...',
    description='Input:',
    layout={'height': '200px', 'width': '90%'}
)
btn = widgets.Button(description="Analisar Treliça", button_style='success')
out = widgets.Output()

def executar(b):
    with out:
        clear_output()
        raw_data = txt_input.value
        try:
            loc = {}
            exec(raw_data, {}, loc)
            # Execução dos cálculos
            f_barras, r_val, r_inf = analisar_trelica(loc['N'], loc['x'], loc['y'], loc['barras'], loc['Fx'],
loc['Fy'], loc['apoios'])

            # Print de estatísticas
            print(f"--- ANÁLISE CONCLUÍDA ---")
            print(f"Força Máxima: {max(np.abs(f_barras)):.2f} N")
            print(f"Reação Vertical Total: {sum(r_val[i] for i, r in enumerate(r_inf) if r[1] == 'y'):.2f}
N")

            plotar_tudo(loc['N'], loc['x'], loc['y'], loc['barras'], f_barras, "Entrada do Usuário")
        except Exception as e:
            print(f"Erro no processamento: {e}\nCertifique-se de que N corresponde ao número
de coordenadas em x e y.")

btn.on_click(executar)
display(txt_input, btn, out)

```

Correção/ código final Gemini

```

import numpy as np
import matplotlib.pyplot as plt

```

```

import pandas as pd
import math
from ipywidgets import widgets
from IPython.display import display, clear_output

# --- 1. MOTOR DE CÁLCULO (IMPLEMENTAÇÃO MANUAL) ---

def resolver_sistema_gauss(A, B):
    """Implementação manual do Método de Gauss"""
    n = len(B)
    M = [A[i] + [B[i]] for i in range(n)]
    for i in range(n):
        max_el = abs(M[i][i])
        max_row = i
        for k in range(i + 1, n):
            if abs(M[k][i]) > max_el:
                max_el = abs(M[k][i]); max_row = k
        M[i], M[max_row] = M[max_row], M[i]
        for k in range(i + 1, n):
            if abs(M[i][i]) < 1e-12: continue
            c = -M[k][i] / M[i][i]
            for j in range(i, n + 1):
                M[k][j] = 0 if i == j else M[k][j] + c * M[i][j]
    x_sol = [0 for i in range(n)]
    for i in range(n - 1, -1, -1):
        if abs(M[i][i]) > 1e-12:
            x_sol[i] = M[i][n] / M[i][i]
            for k in range(i - 1, -1, -1):
                M[k][n] -= M[k][i] * x_sol[i]
    return x_sol

def analisar_e_exibir(N, x, y, barras, Fx, Fy, apoios, titulo):
    """Realiza o processamento completo e gera as saídas"""
    num_barras = len(barras)
    reacoes = []
    for i, tipo in enumerate(apoios):
        if tipo == 'fixo': reacoes.extend([(i, 'x'), (i, 'y')])
        elif tipo == 'movel_y': reacoes.append((i, 'y'))
        elif tipo == 'movel_x': reacoes.append((i, 'x'))

    total_incognitas = num_barras + len(reacoes)
    A = [[0.0 for _ in range(total_incognitas)] for _ in range(2 * N)]
    B_vec = [0.0 for _ in range(2 * N)]

    for i, (n1, n2) in enumerate(barras):
        dx, dy = x[n2] - x[n1], y[n2] - y[n1]
        L = math.sqrt(dx**2 + dy**2)
        cos, sen = dx / L, dy / L

```

```

A[2*n1][i] += cos; A[2*n1+1][i] += sen
A[2*n2][i] -= cos; A[2*n2+1][i] -= sen

for j, (no, eixo) in enumerate(reacoes):
    col = num_barras + j
    A[2*no + (0 if eixo == 'x' else 1)][col] = 1.0

for i in range(N):
    B_vec[2*i], B_vec[2*i+1] = -Fx[i], -Fy[i]

sol = resolver_sistema_gauss(A, B_vec)
forcas = sol[:num_barras]

# --- SAÍDAS ---
print(f"\n{'-'*30}\nANÁLISE: {titulo}\n{'-'*30}")

# Tabela de Resultados
df = pd.DataFrame({
    'Elemento': [f"Barra {i} ({barras[i][0]}-{barras[i][1]})" for i in range(num_barras)],
    'Força (N)': [round(f, 2) for f in forcas],
    'Status': ['Tração (+)' if f > 0.1 else 'Compressão (-)' if f < -0.1 else 'Nula' for f in forcas]
})
print(df.to_string(index=False))

# Estatísticas
print(f"\nEstatísticas:")
print(f"        Barra mais solicitada: Barra {np.argmax(np.abs(forcas))}
({max(np.abs(forcas)):.2f} N)")
print(f"        Força média: {np.mean(np.abs(forcas)):.2f} N")

# Gráficos
fig, axs = plt.subplots(2, 2, figsize=(16, 10))
fig.suptitle(f"Resultados Visuais - {titulo}", fontsize=16)

# 1. Desenho Original e Apoios
for i, (n1, n2) in enumerate(barras):
    axs[0,0].plot([x[n1], x[n2]], [y[n1], y[n2]], 'k--', alpha=0.3)
axs[0,0].scatter(x, y, color='black', s=100)
for i in range(N):
    axs[0,0].text(x[i], y[i], f"Nó {i}", fontsize=12, color='darkblue', fontweight='bold')
axs[0,0].set_title("1. Geometria Original e Numeração")

# 2. Treliza Colorida (Mapa de Intensidade)
max_f = max(np.abs(forcas)) if max(np.abs(forcas)) > 0 else 1
for i, (n1, n2) in enumerate(barras):
    f = forcas[i]
    cor = 'blue' if f > 0.1 else 'red' if f < -0.1 else 'gray'
    lw = 2 + 5 * (abs(f)/max_f)

```

```

        axs[0,1].plot([x[n1], x[n2]], [y[n1], y[n2]], color=cor, linewidth=lw)
        axs[0,1].text((x[n1]+x[n2])/2, (y[n1]+y[n2])/2, f'{f:.0f}', fontsize=9,
backgroundcolor='white')
        axs[0,1].set_title("2. Esforços Internos (Azul: Tração | Vermelho: Compr.)")

# 3. Gráfico de Magnitudes
axs[1,0].bar([f"{b[0]}-{b[1]}" for b in barras], [abs(f) for f in forcas], color='teal')
axs[1,0].set_title("3. Magnitude Absoluta por Barra")
axs[1,0].set_ylabel("Força |F| (N)")

# 4. Histograma
axs[1,1].hist(forcas, bins=10, color='orange', edgecolor='black', alpha=0.7)
axs[1,1].axvline(0, color='red', linestyle='--')
axs[1,1].set_title("4. Distribuição das Forças Internas")

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

```

--- 2. EXECUÇÃO AUTOMÁTICA DA TRELIÇA WARREN ---

```

# Dados da Warren (4 nós, 5 barras, simétrica)
# Nó 0(0,0), Nó 1(4,0), Nó 2(2,3.46), Nó 3(2,0) -> Erro no seu enunciado: Nó central inferior
precisa existir
# Vamos definir: 0(esq), 1(dir), 2(meio baixo), 3(meio alto)
w_N = 4
w_x = [0, 4, 2, 2]
w_y = [0, 0, 0, 3] # Nó 2 é o central inferior
w_barras = [[0, 2], [2, 1], [0, 3], [1, 3], [2, 3]]
w_Fx = [0, 0, 0, 0]
w_Fy = [0, 0, -1000, 0] # Carga de 1000N no nó central inferior
w_apoios = ['fixo', 'movel_y', 'livre', 'livre']

analisar_e_exibir(w_N, w_x, w_y, w_barras, w_Fx, w_Fy, w_apoios, "Treliza Warren
(Referência)")

```

--- 3. INTERFACE PARA O USUÁRIO (COLE SEUS DADOS) ---

```

print("\n" + "="*50)
print("AGORA: COLE OS DADOS DA SUA TRELIÇA PERSONALIZADA ABAIXO")
print("="*50)

txt_input = widgets.Textarea(
    placeholder='Cole aqui seu script de input (N=..., x=..., etc)',
    layout={'height': '200px', 'width': '95%'}
)

btn = widgets.Button(description="Analisar Input", button_style='primary')
out = widgets.Output()

```

```

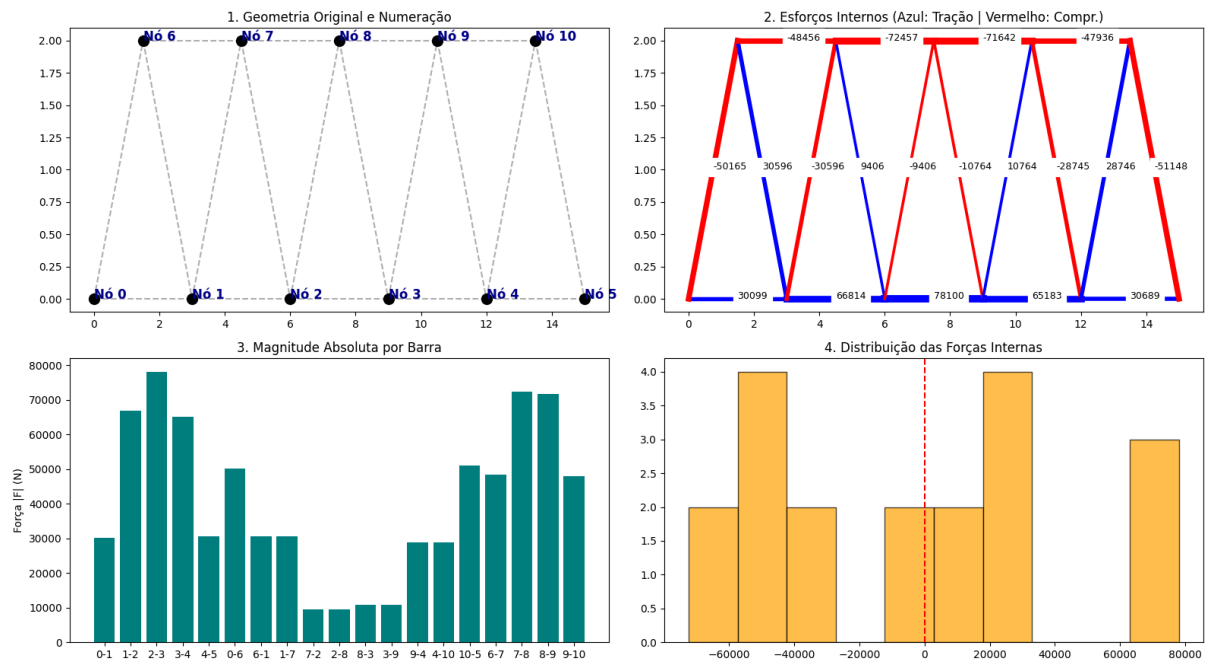
def processar_custom(b):
    with out:
        clear_output()
        try:
            local_vars = {}
            exec(txt_input.value, {}, local_vars)
            analisar_e_exibir(
                local_vars['N'], local_vars['x'], local_vars['y'],
                local_vars['barras'], local_vars['Fx'], local_vars['Fy'],
                local_vars['apoios'], "Treliza do Usuário"
            )
        except Exception as e:
            print(f"Erro: {e}. Verifique se as listas x, y, Fx, Fy e apoios têm o mesmo tamanho
N.")

btn.on_click(processar_custom)
display(txt_input, btn, out)

```

Anexos Gemini

Resultados Visuais - Treliza do Usuário



Resultados Visuais - Treliza Warren (Referência)

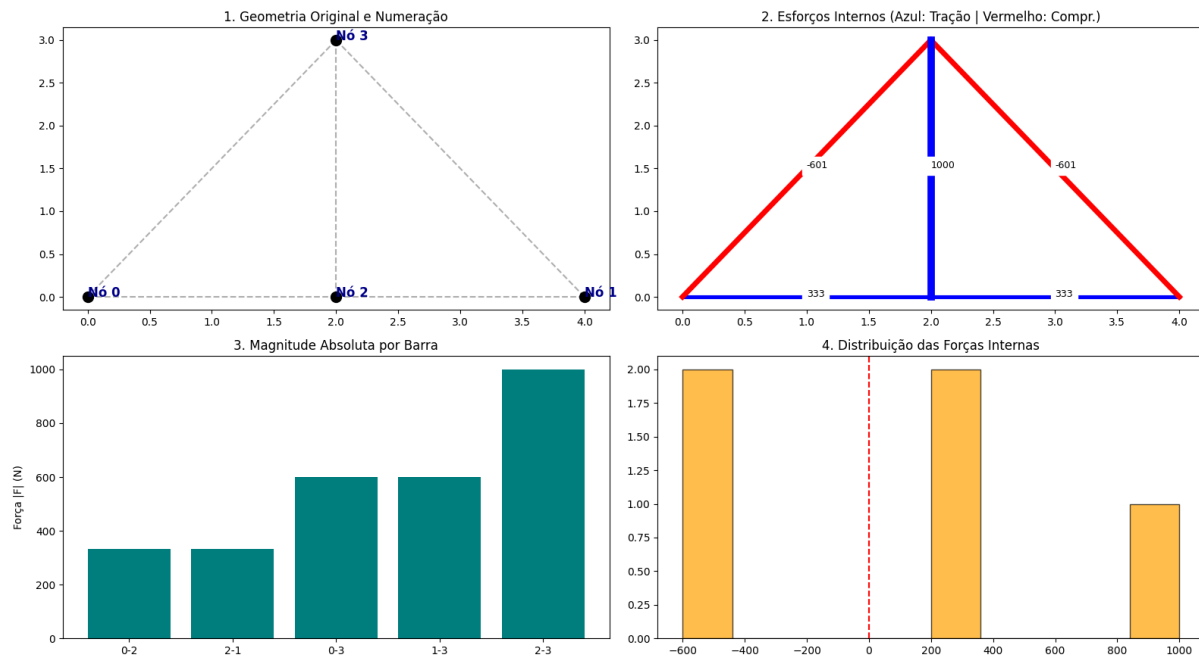


Tabela de descrição

Chat GPT

Critério	Descrição
Corretude numérica	Para os valores que testei (gerados pelo tester paralelo que pedi para criarem também), os valores estavam bons.
Robustez	Precisei alterar várias vezes todos (vou descrever melhor na comparação final) para eles entregarem valores que funcionassem.
Qualidade dos gráficos	Bons. O Chat GPT me deu 4 gráficos para a treliza do input do usuário e mais 4 gráficos para a treliza teste (treliza warren)
Eficiência	Segundo ele mesmo: “Muito rápido em Colab/computador comum. Para $N \approx 32$ nós, normalmente execução em frações de segundo até poucos segundos, dependendo da quantidade de barras.”
Qualidade do código	Código bem comentado, modularizado em funções e alguns tratamentos de erro.
Aderência ao prompt	Implementou gauss jordan manualmente,

	mas sobre “aderência ao prompt” geral, precisei pedir para mudar algo que eu já havia mencionado várias vezes!
--	--

Gemini

Critério	Descrição
Corretude numérica	Bons valores.
Robustez	Segundo ele mesmo: “Sim. O código utiliza listas dinâmicas e monta a matriz de rigidez global $2N \times (B+R)$ baseada estritamente nos inputs fornecidos”
Qualidade dos gráficos	Gosto dos gráficos do Gemini. Estes foram bons sim.
Eficiência	Segundo ele: “Sub-segundo ($< 0.1s$). A eliminação de Gauss manual tem complexidade algorítmica $O(n^3)$. Para $N=32$, o sistema tem aproximadamente 64 equações, o que é processado instantaneamente em qualquer ambiente moderno como o Colab”
Qualidade do código	Me parece um código bem legível, mas precisa de bastante foco para as áreas onde ele implementa coisas matemáticas e figuras(várias descrições).
Aderência ao prompt	Resolveu manualmente Gauss. Como as outras LLM's, precisei orientar mais algumas vezes além da primeira para atingir o resultado que eu queria.

Copilot

Critério	Descrição
Corretude numérica	Semelhante aos outros.
Robustez	Segundo ele: “Sim, funciona para $N = 4, 8, 16, 32$ sem alterações. O código usa listas de tamanho variável e monta a matriz de equilíbrio dinamicamente, então escala bem para esses valores “
Qualidade dos gráficos	Bem legíveis também.
Eficiência	Segundo ele: “Para $N = 32$ (64 equações),

	o tempo de execução é praticamente instantâneo em Colab (< 1 segundo)”
Qualidade do código	Eu nunca havia usado o Copilot antes. Achei o código enxuto, mas me parece bom. Só achei um pouco difícil a leitura de algumas funções.
Aderência ao prompt	Implementou Gauss manualmente também, mas precisei repetir algumas coisas vááárias vezes.

Opinião e comparação final das 3 LLM's

Antes de tudo, pedi para o Chat GPT criar um software paralelo “Tester” (não sei se é assim que deve se chamar), para que ele sempre me gerasse valores diferentes, aleatórios e factíveis para eu usar de input de novas treliças (visto que sempre que eu quisesse colocar um input, não conheceria muito bem dados de treliças). Com isso, os testes dos códigos gerados por ela são com dados reais gerados por um software paralelo que gerasse valores para as treliças do usuário.

Nos códigos principais, depois de algumas correções (pedindo novamente algumas coisas que eu já havia escrito!), os 3 resultados foram bons. Foi minha primeira vez testando o Copilot. Em todos houveram problemas no primeiro código gerado. Primeiro código do Chat GPT: não dava a opção do usuário colocar um input. Pedi então que houvesse uma área para colar os dados da treliça do usuário. Primeiro código do Gemini: Obtive um erro pois o input que o Gemini fez não aceitava os dados. Depois ele corrigiu, e tive que pedir para ele gerar os gráficos da treliça warren também. Primeiro código do Copilot: também estava com input fixo, não dando espaço para o usuário colocar os dados. Depois de corrigido isso, a outra versão deu erro (era algo que creio que um try-except resolvesse). Ainda dando erro depois, pedi que o Copilot revisasse os dados da treliça de comparação (warren) que estava dando erro. Depois de muito explicar coisas, deu certo.

No geral, o resultado de todos funcionou, apesar de por exemplo o Copilot ter demandado mais paciência. O Gemini achei bem interessante. O Chat GPT (por eu estar mais acostumado) não me surpreendeu.